



Model Context Protocol (MCP)

From Buzzword to Build-Ready

SkillUp AI Workshop by

Pankaj Aher & Tezan Sahu

A little about us...



Pankaj Aher is a Principal Software Engineer at Microsoft IDC. He leads transformative efforts in Copilot extensibility—most notably around Substrate Search Service (3S), Model Context Protocol (MCP), dynamic tool schemas, and authentication frameworks. With two US patents to his name, Pankaj exemplifies innovation grounded in real-world impact

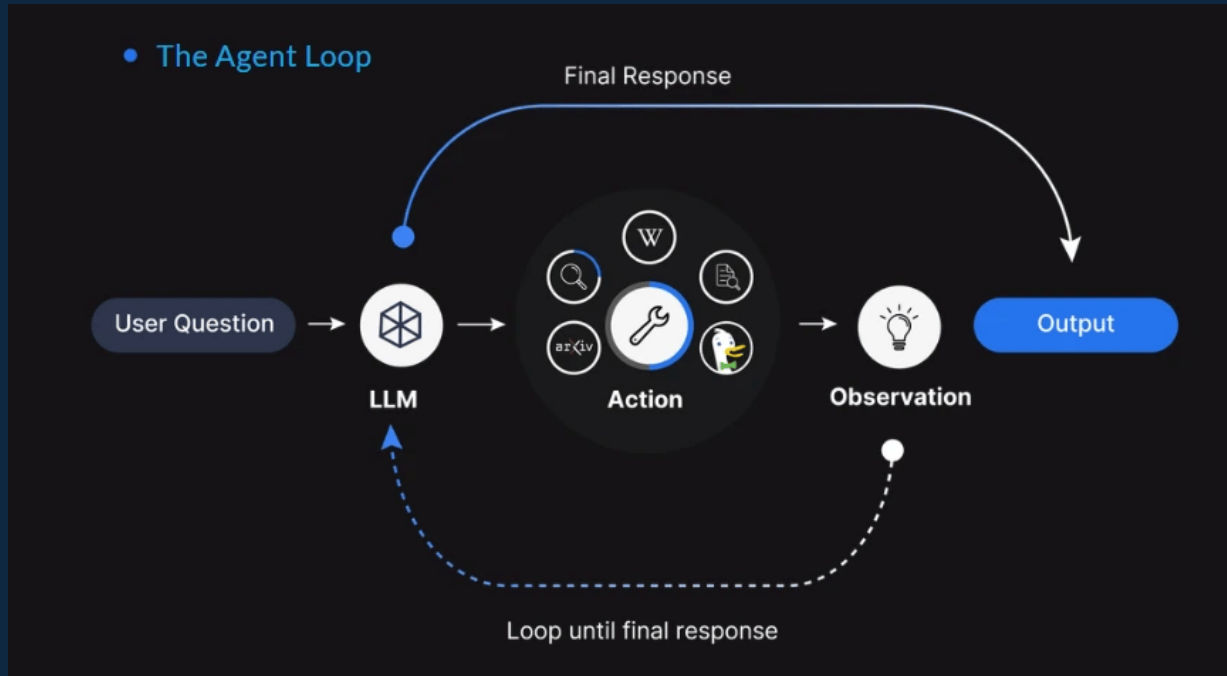
Tezan Sahu is a Software Engineer II at Microsoft IDC (building Enterprise Copilot Extensibility experiences), previously a Data & Applied Scientist at the company. An IIT Bombay graduate, he's the author of the national bestseller “Beyond Code” and holds two US patents in applied AI. He also mentors professionals and students in AI, data science, and product-building on Topmate.



Agenda

- AI Agents & Tool Use
- Introduction to MCP
- MCP Architecture
- MCP Primitives (with demos)
- Ecosystem Overview – MCP Servers that you can use
- **Demos**
 - Creating an M365 Copilot Agent with MCP Tools using Agent Toolkit (ATK)
 - Creating an M365 Copilot Agent with Remote MCP Servers using Copilot Studio
 - Using MCP Servers with GitHub Copilot in VS Code
- Security in MCP – Threats & mitigation strategies
- Q&A

What's an AI Agent?



An AI agent is a **system** or program capable of **autonomously performing tasks** on behalf of a user or another system by **designing its workflow** and **utilizing available tools**

LLMs-Based AI Agents

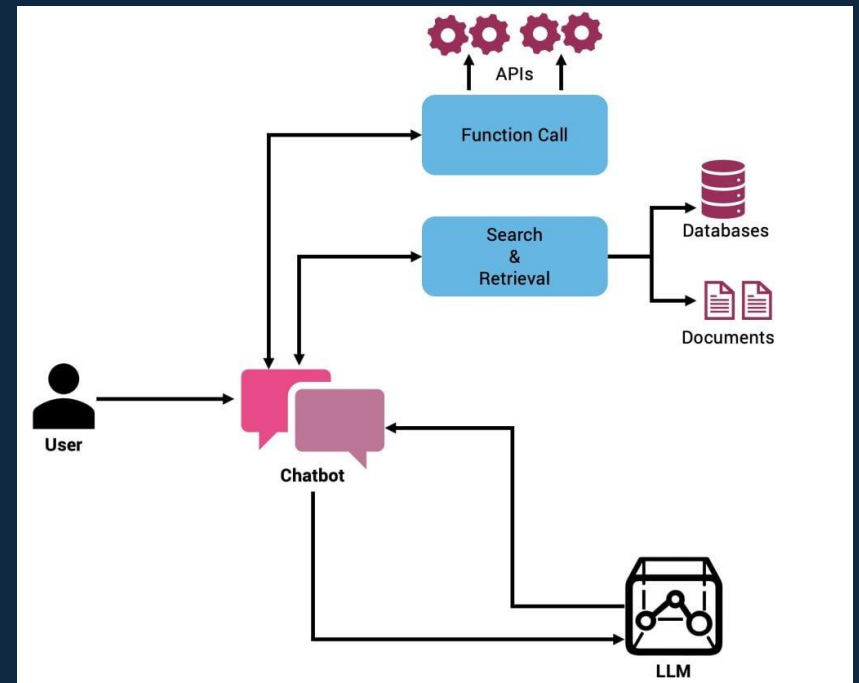
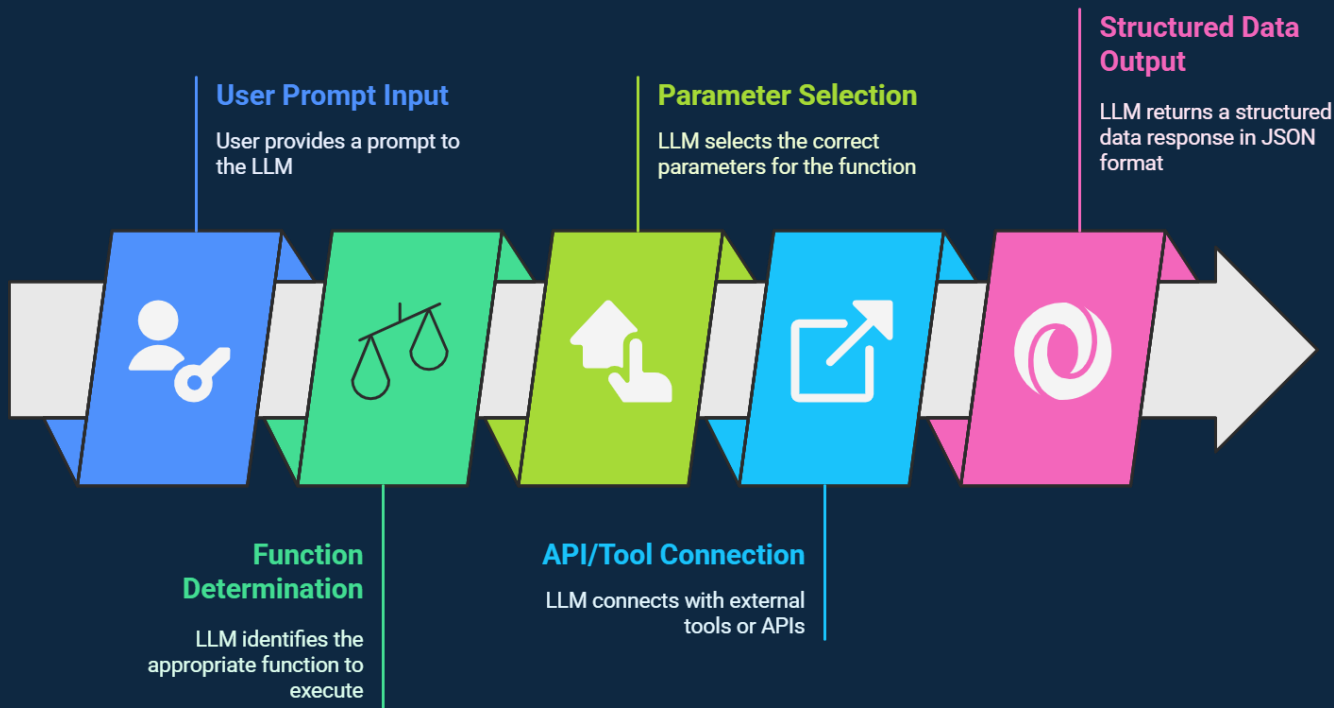
- 🧠 Large Language Models (LLMs) act as the core reasoning engine
- 🧠 Provide advanced reasoning, planning & synthesis capabilities
- 🧠 Can generalize from examples and handle novel tasks with remarkable proficiency

Limitations

- 🚫 Lacks real-time knowledge (cutoff dates)
- 🚫 Prone to hallucination (confident but incorrect)
- 🚫 No direct interaction with external systems

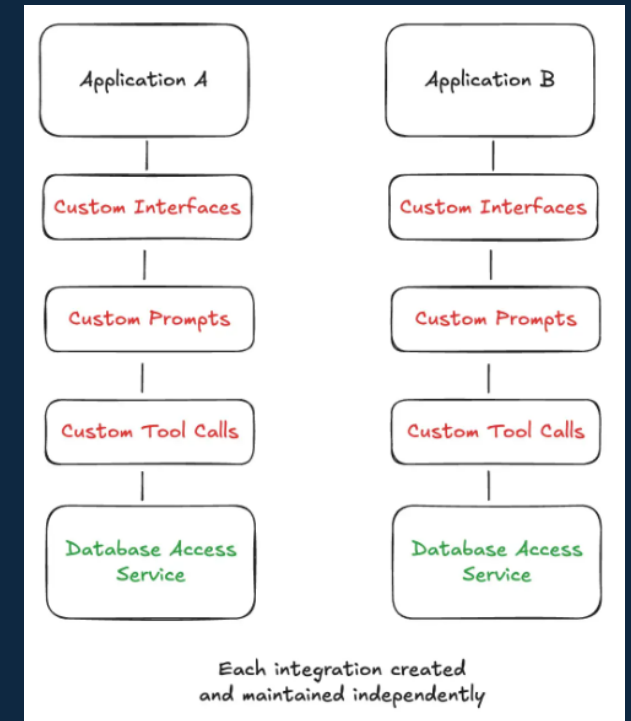
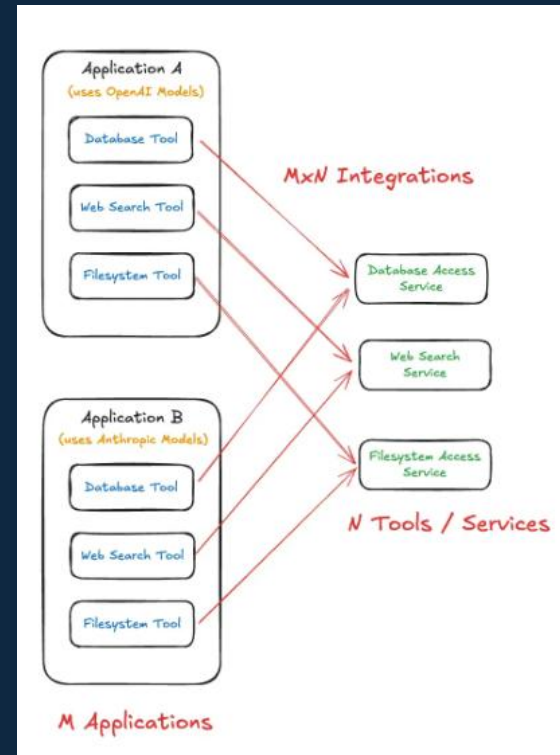
Tool Use in LLM-Powered Agents

LLM's ability to determine, from a user prompt, the **appropriate function to execute** from available options and the **correct parameters** to pass to that function



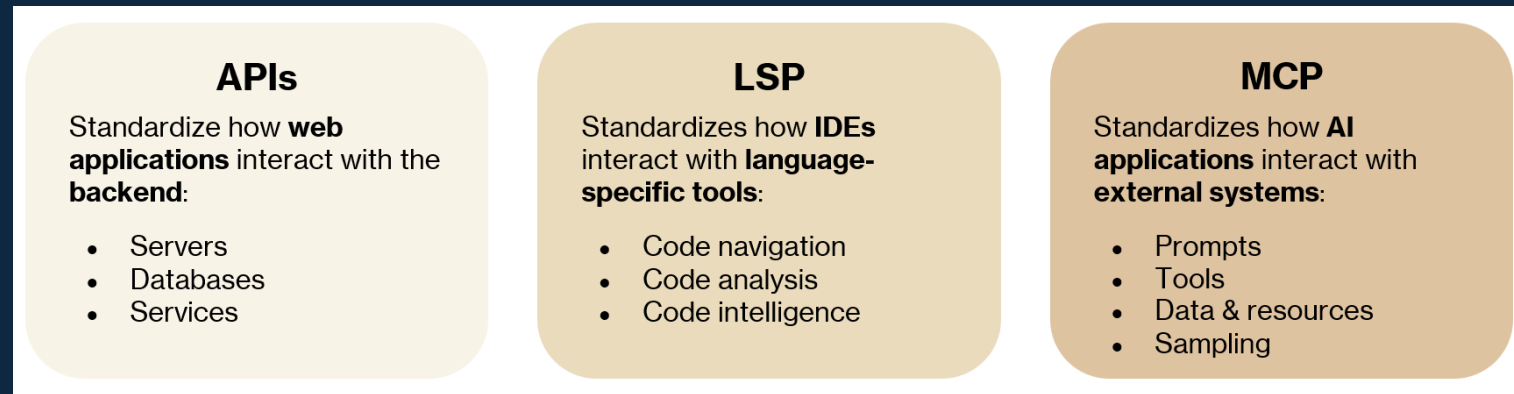
The MxN Problem for AI System Integration

- **Fragmentation explosion**
 - With **M AI Apps** (e.g. ChatGPT, Claude, Cursor) and **N tools** (e.g. calendar API, DB connector), naive teams build $M \times N$ custom integrations
 - Example: 2 app + 3 tools = 6 connections
 - Hard to maintain
- **Pain points:**
 - Duplicated engineering
 - Maintenance overhead
 - Inconsistent interfaces & behavior
 - Limited Scalability
- Leads to demand for a universal tool-calling standard that decouples tools from models.

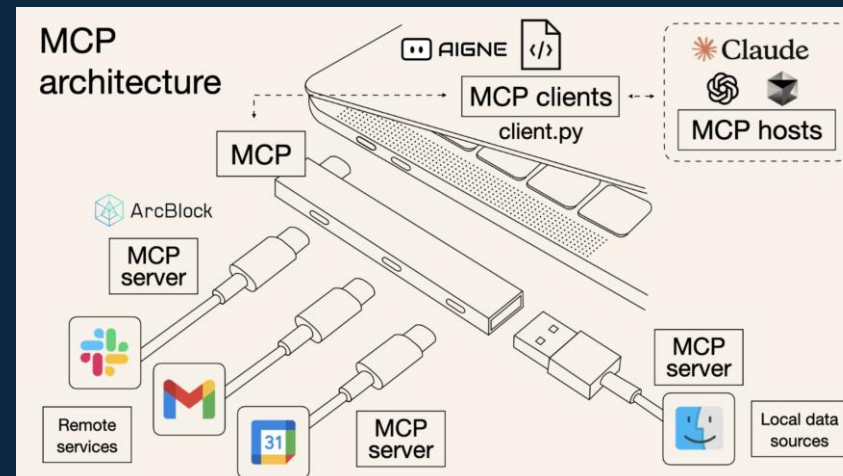


Enter “Model Context Protocol (MCP)”

- What is MCP?
 - Open **Protocol** for providing **Context** (through tools & data sources) to **Models** in AI Apps

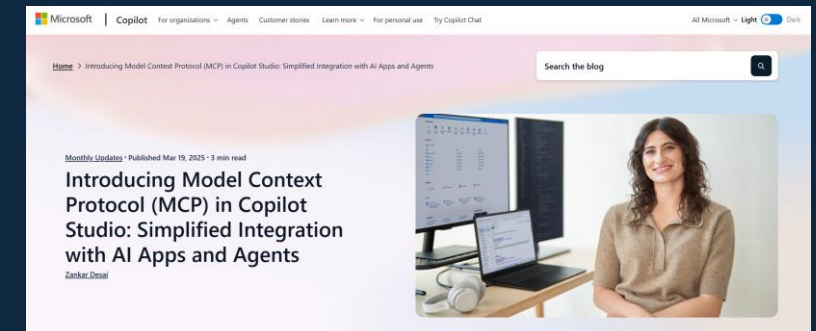
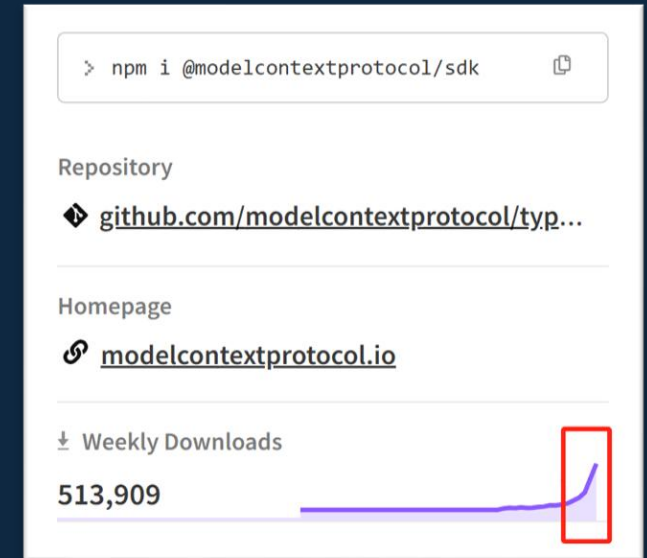


- Introduced by **Anthropic**
- **Type C of Tools: 100% Reusability**



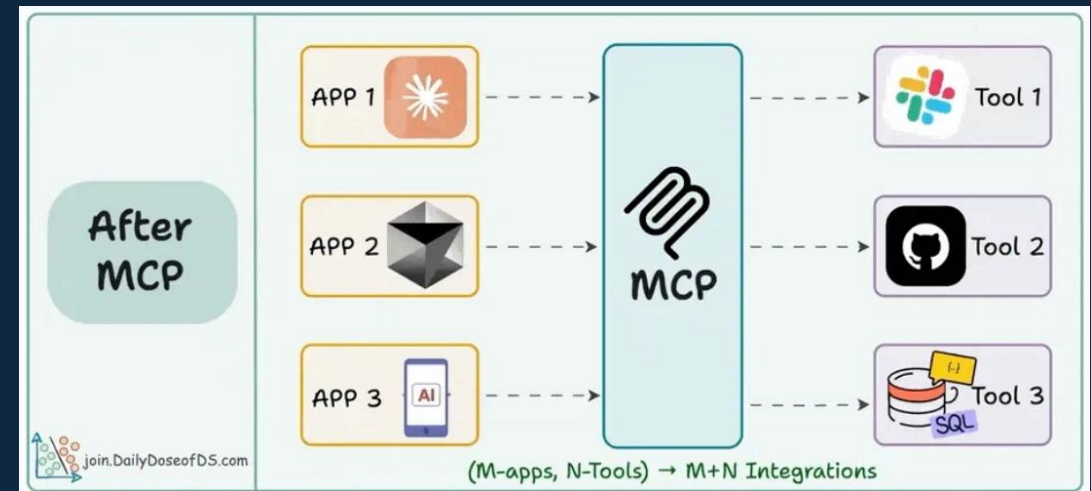
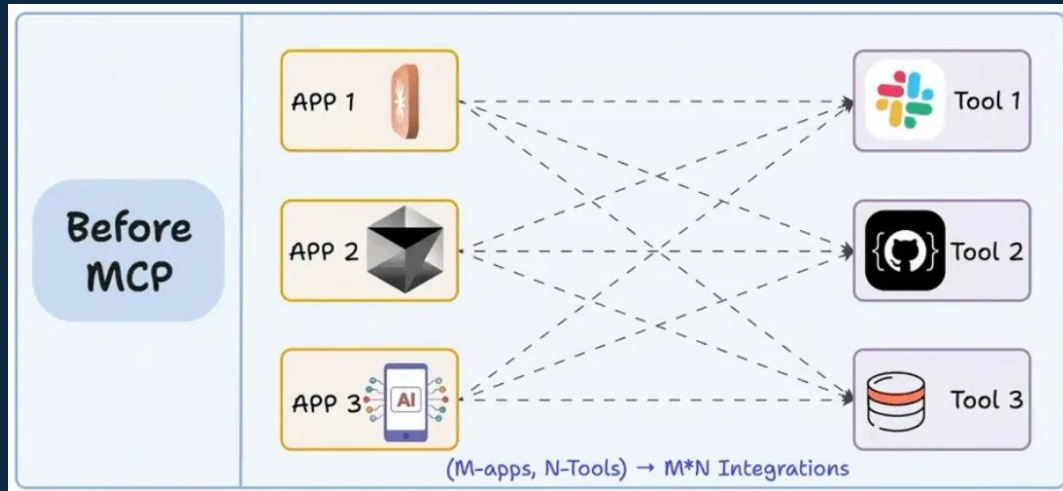
Enter “Model Context Protocol (MCP)”

- Timeline & Trends
 - **Nov 2024** - Anthropic announced MCP, Zed supported MCP
 - **Dec 2024** - Cline supported MCP
 - **Jan 2025** - Cursor supported MCP
 - **Feb 2025** - Windsurf supported MCP
 - Meteoric rise as developers recognize integration potential
 - **Enterprise Adoption:** Block, Apollo, Microsoft Azure & Copilot Studio



Core Value Proposition of MCP

MCP transforms the **MxN** integration problem into a more manageable **M+N** solution

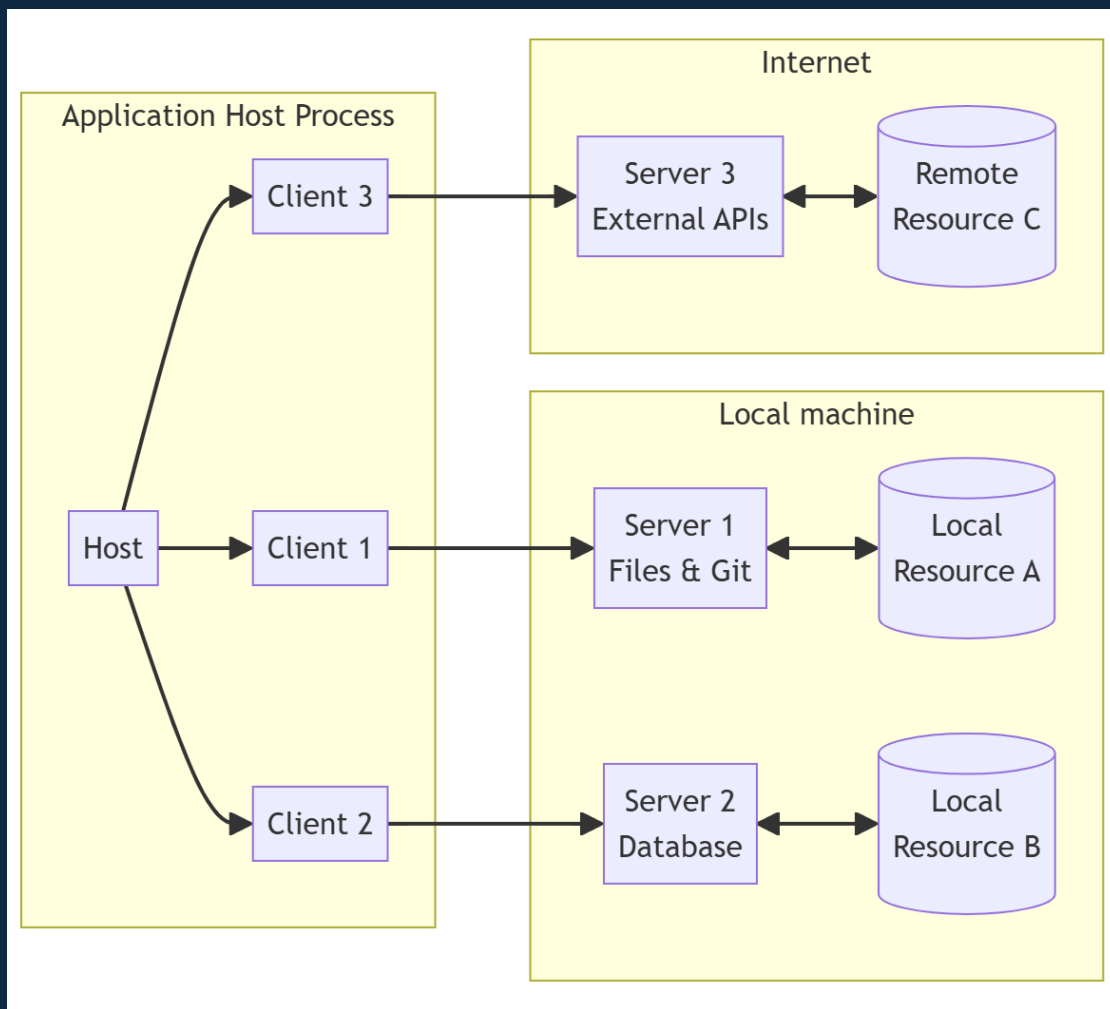


- **For AI Application developers**
 - Connect your app to any MCP server with 0 additional work
- **For Tool or API developers**
 - Build an MCP server once, see adoption everywhere

Clarifying MCP Misconceptions

- **What MCP Is NOT** – Common Misconceptions
 - Not an Orchestrator
 - Not a Middleware Platform
- **What MCP Actually Is** – A Protocol (*a set of rules*)
 - **Standardized Communication:** Common language for AI-tool interaction
 - **Transport Agnostic:** Works over various connection types
 - **JSON-RPC Based:** Uses established standards for 2-way communication
 - **Capability Negotiation:** Dynamic discovery of available functions

MCP Architecture – Core Components



- **MCP Hosts**

- User-facing AI application that want to access data through MCP
- Programs like Claude Desktop, Visual Studio Code, IDEs, or AI tools

- **MCP Clients**

- Maintain 1:1 stateful connections with servers
- Manages 2-way communication details

- **MCP Servers:**

- Lightweight programs that each expose specific capabilities through the standardized MCP
- Types: Local, Remote

MCP Server Primitives

- **Tools** 🛠️

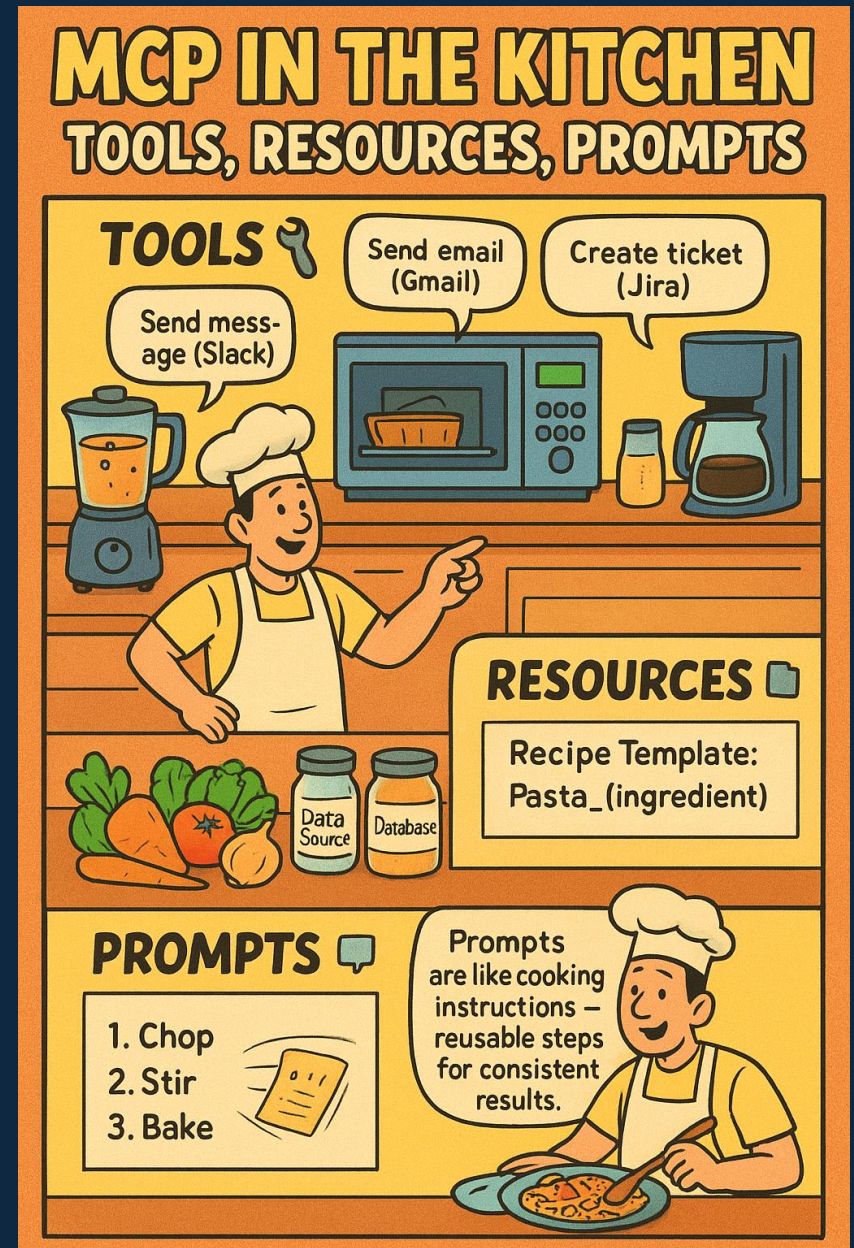
- Executable functions that AI models can invoke to perform actions with potential side effects
- *Simply put, “They do the work”*
- Example: Slack (send message), Gmail (send email), Jira (create ticket)

- **Resources** 📄

- Read-only access to data sources allowing AI models to retrieve context without complex logic execution
- *Simply put, “This is the information you can consume”*
- Resource Templates: Dynamic resource generation with parameterized URIs

- **Prompts** 💬

- Reusable prompt templates that provide structured interaction patterns for AI models
- *Simply put, “They are templates for consistent results”*



MCP Server Primitives

```
{
  "name": "send_email",
  "description": "Send an email message",
  "parameters": {
    "type": "object",
    "properties": {
      "to": {"type": "string"},
      "subject": {"type": "string"},
      "body": {"type": "string"}
    },
    "required": ["to", "subject", "body"]
  }
}
```

Sample Tool

```
{
  "uri": "file:///project/README.md",
  "name": "README.md",
  "title": "Project Documentation",
  "mimeType": "text/markdown",
  "text": "Project overview and setup instructions..."
}
```

Sample Resource

```
{
  "uriTemplate": "weather://{city}/current",
  "name": "Weather Data",
  "description": "Current weather for specified city"
}
```

Sample Resource Template

```
{
  "name": "customer_issue_summary",
  "description": "Generate customer support summary",
  "parameters": {
    "customer_id": {"type": "string"},
    "time_span": {"type": "string"}
  },
  "template": "Analyze customer {customer_id} issues over {time_span}..."
}
```

Sample Prompt

Time for some Live Action!



Demo – Spinning Up MCP Servers

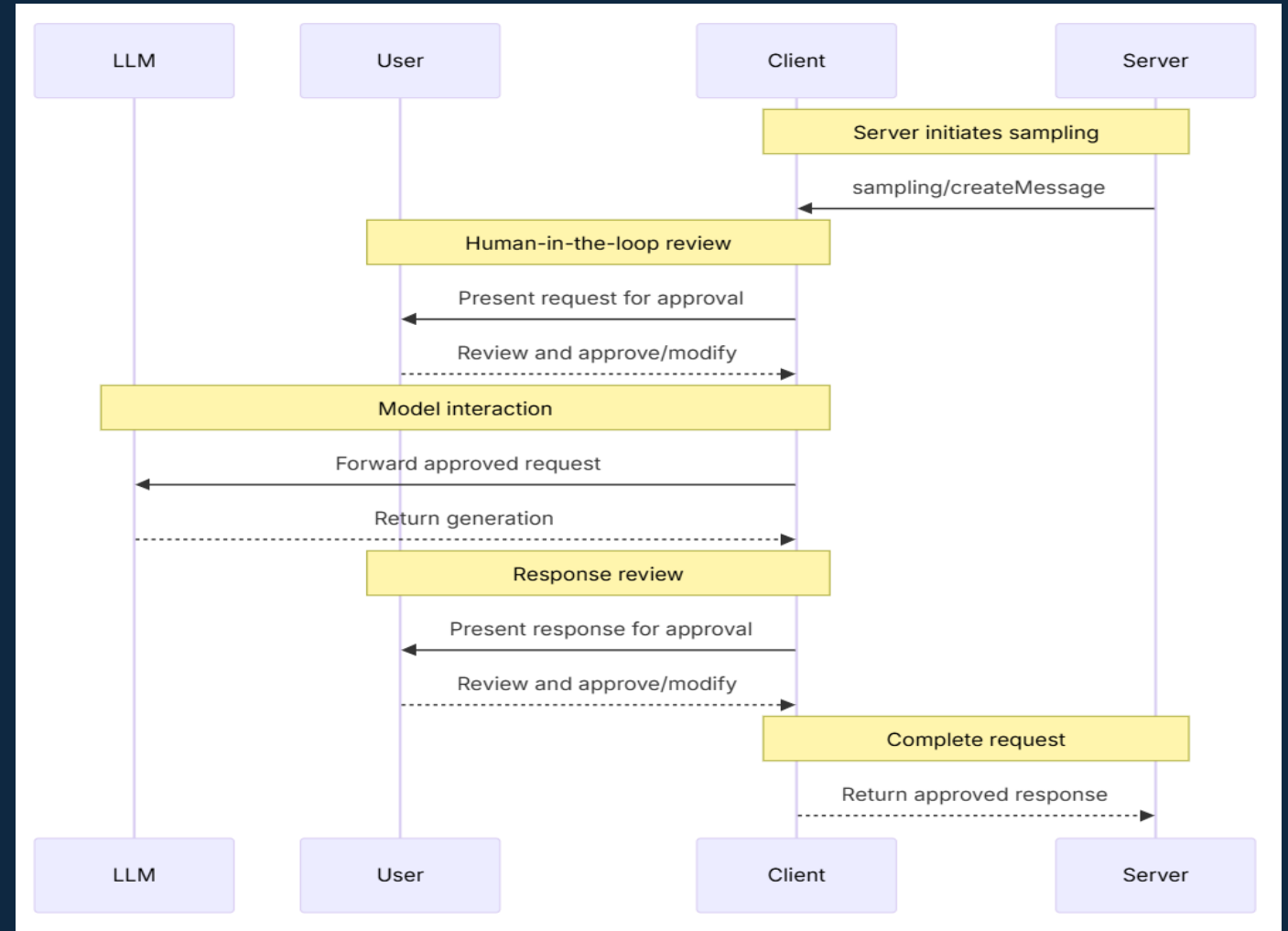
Build an MCP server for Asset Management

- Scaffold with GitHub Copilot
- Implement MCP primitives: **Tools, Prompts, Resources**
- Validate end-to-end in **VS Code** and **MCP Inspector**
- **Resources:** <https://modelcontextprotocol.info/docs/tutorials/building-mcp-with-llms/>

MCP Client Primitives

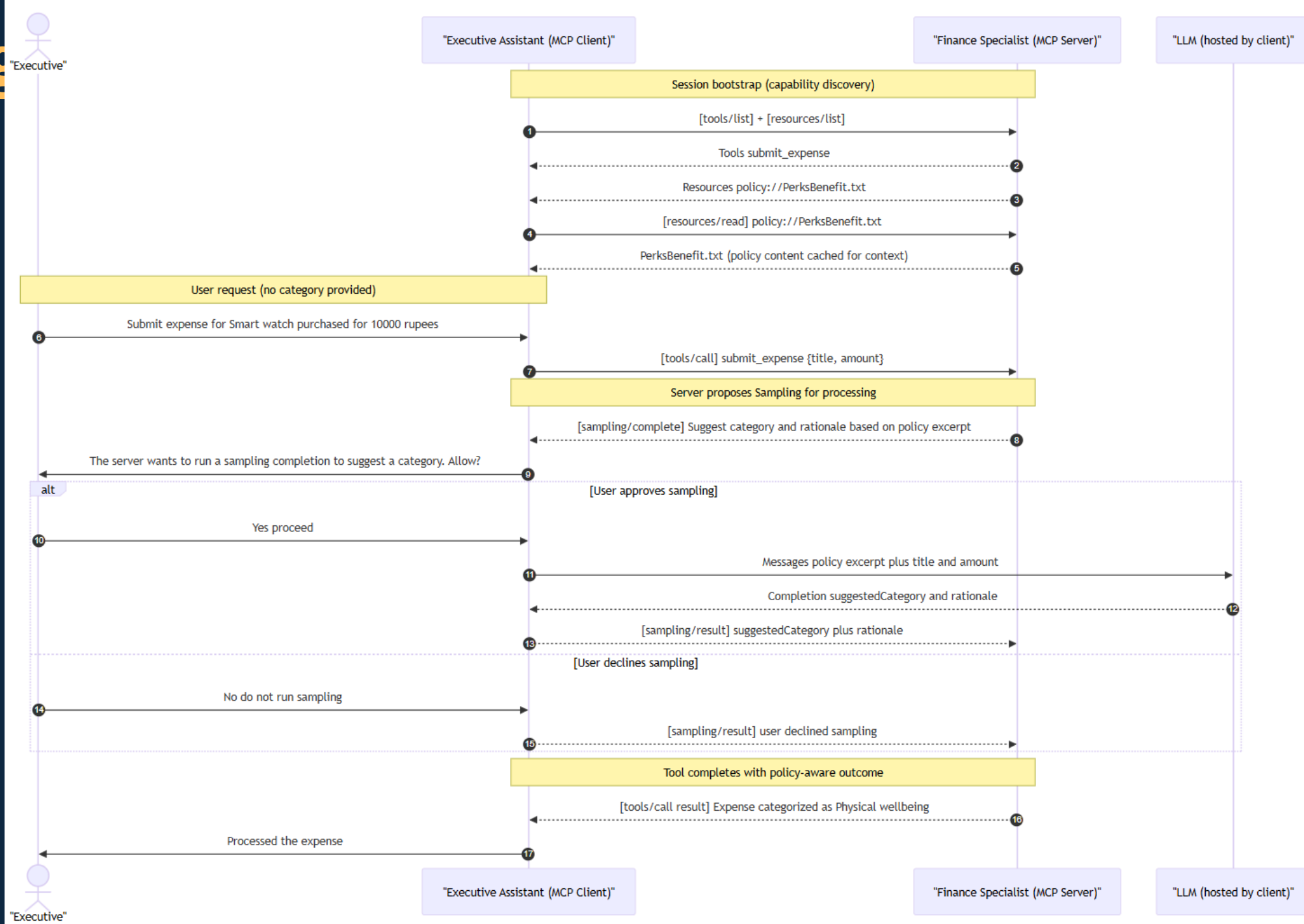
Sampling

- Allows servers to request language model completions from the client's AI application
- Enables server logic to offload generative tasks to the client's trusted LLM access—without needing API keys or direct integration



"Sampling" – Technical Sequence Diagram

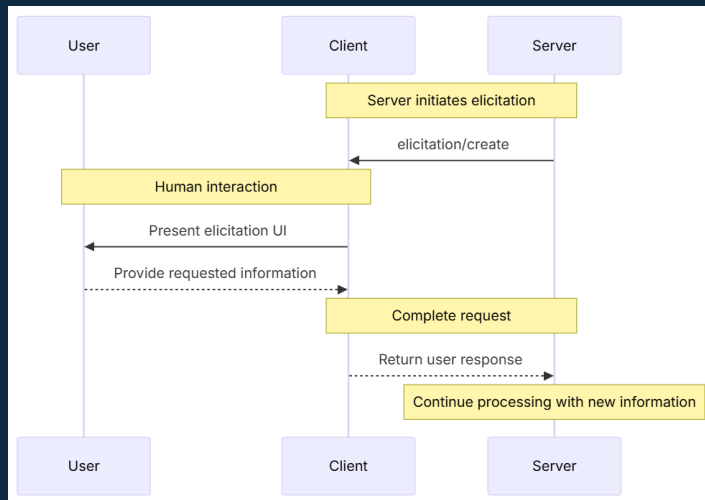
Sampling example



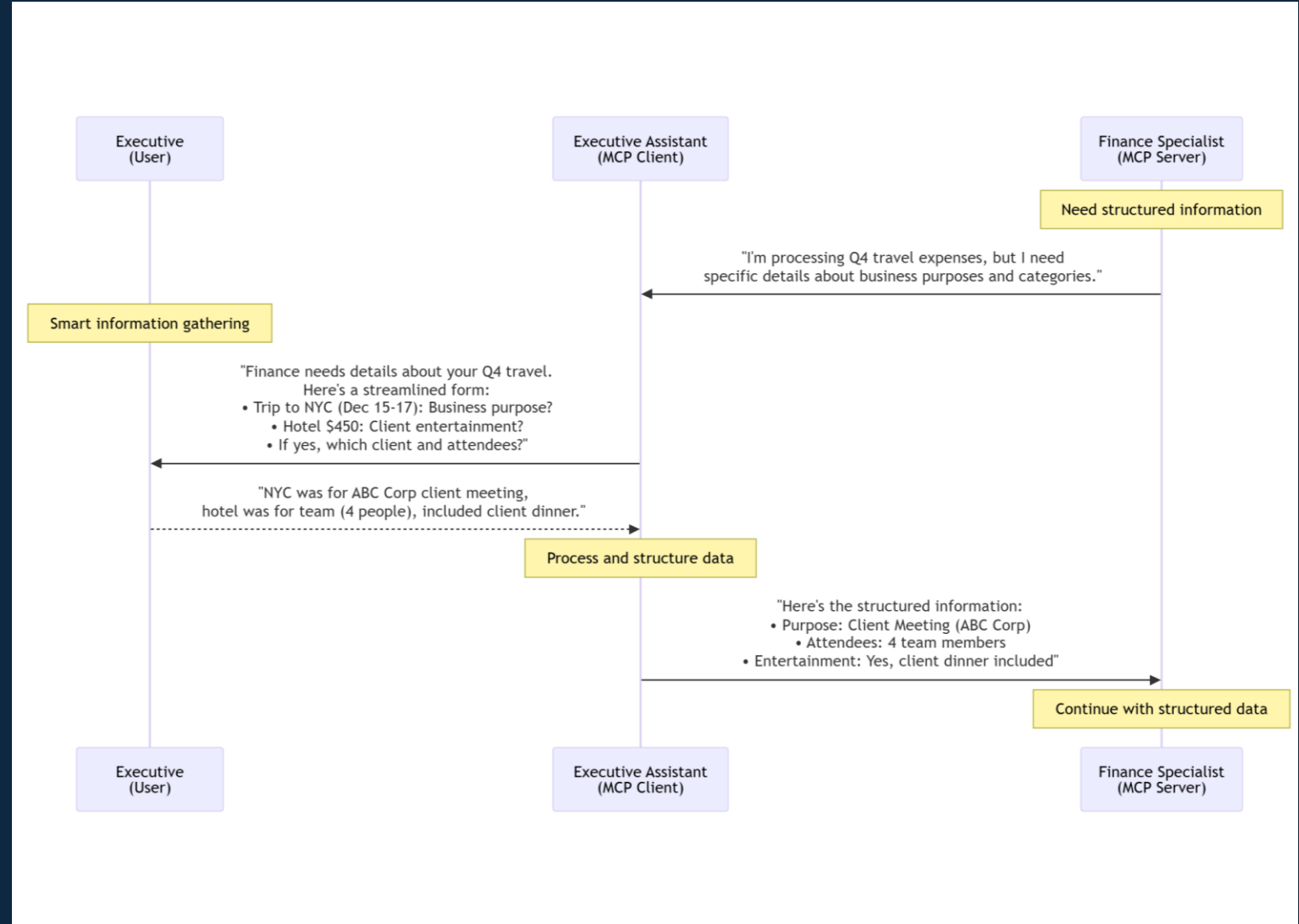
MCP Client Primitives

Elicitation

- Enables server to request structured information (like user choices, emails, numbers) during a workflow
- Client presents a UI prompt, collects data according to a provided [restricted JSON Schema], and sends a response back.



“Elicitation” – Technical Sequence Diagram



“Elicitation” – Executive Assistant Analogies

Time for some Live Action!



Demo – Spinning Up MCP Servers

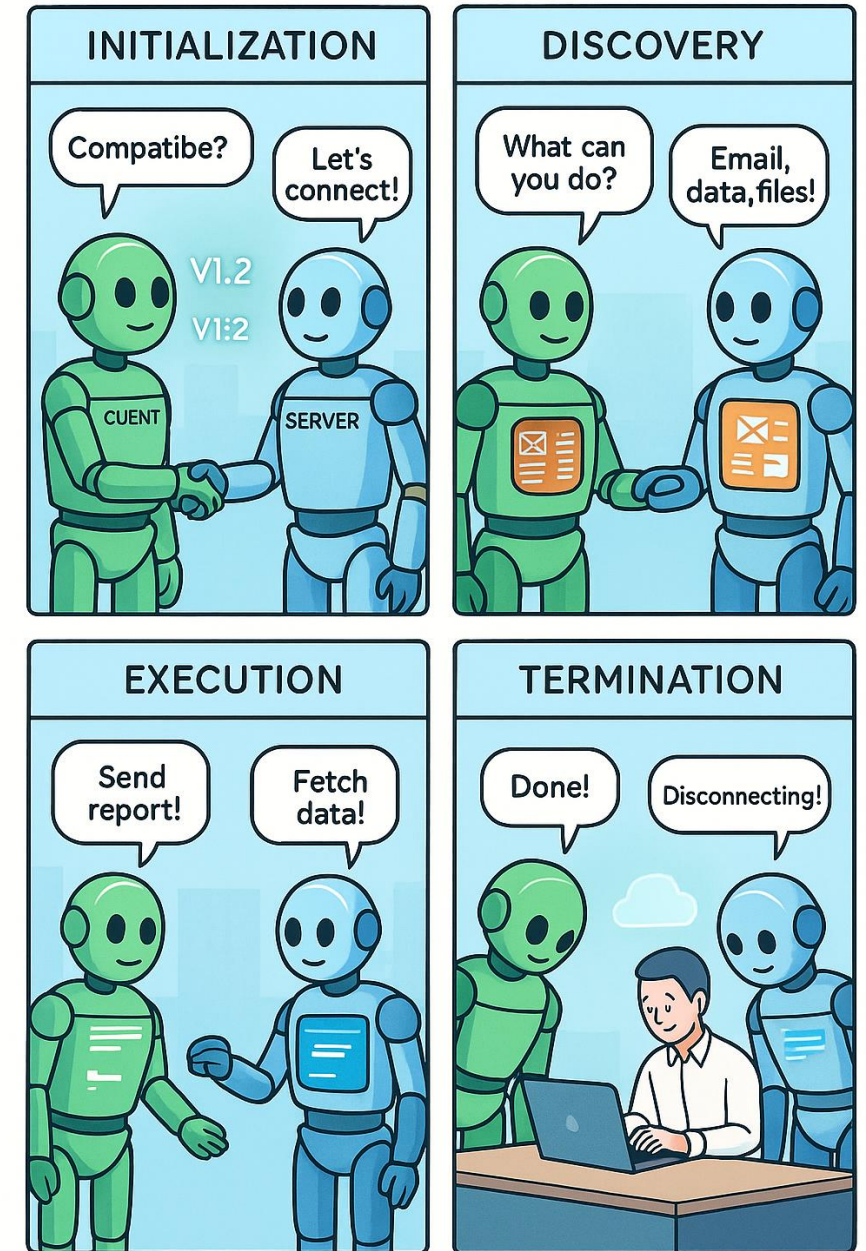
Build an MCP server for Asset Management

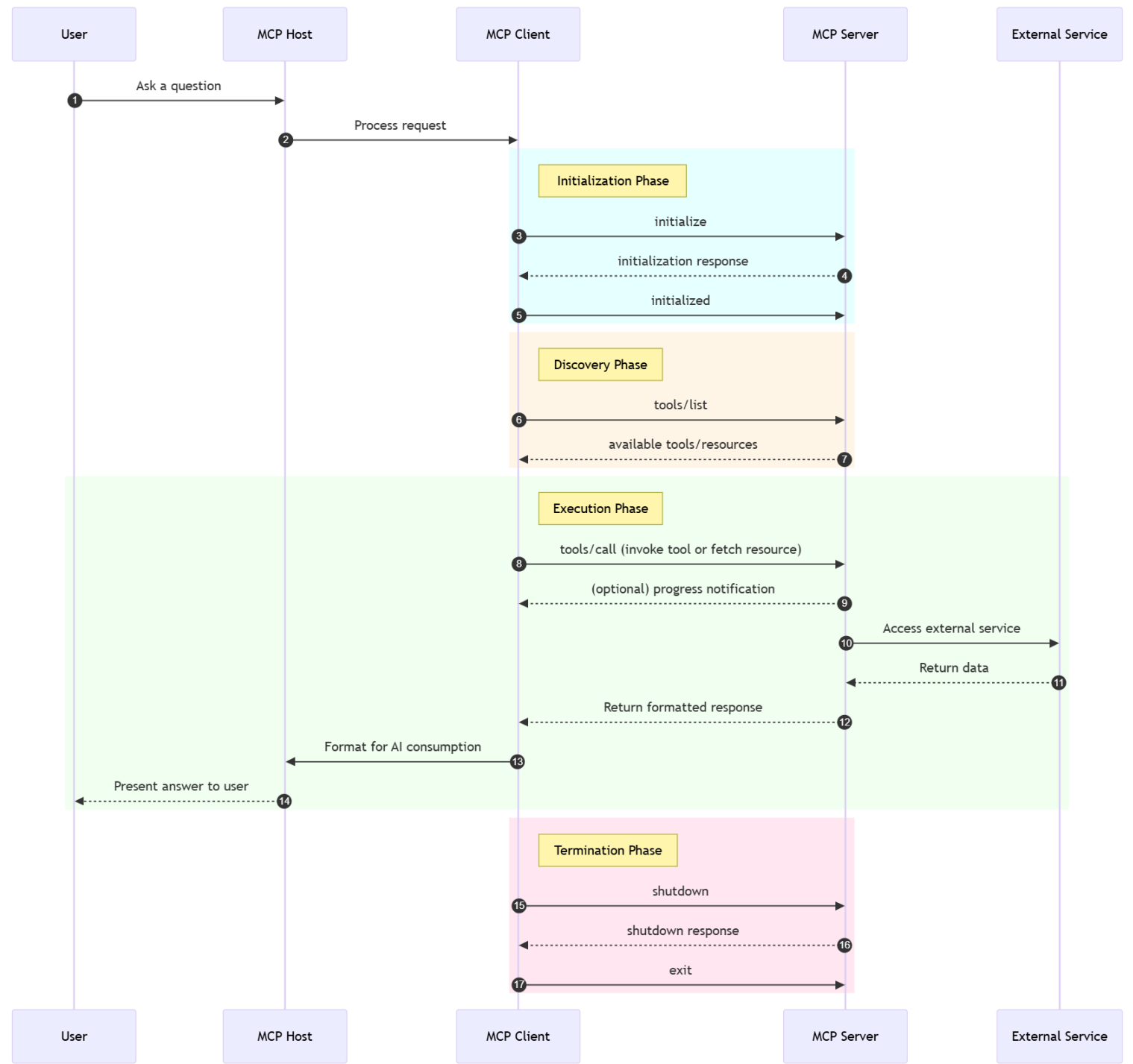
- Scaffold with GitHub Copilot
- Implement MCP primitives: **Tools, Prompts, Resources**
- Validate end-to-end in **VS Code** and **MCP Inspector**
- Add **Elicitation** and **Sampling** capabilities
- Integrate **OAuth** for secure access

Interaction Lifecycle

4 Stages

- **Initialization**
 - Client connects to the server, and they exchange protocol versions and capabilities
- **Discovery**
 - Client requests information about available capabilities
 - Server responds with a list of available tools
- **Execution**
 - Client invokes capabilities based on the Host's needs
- **Termination**
 - Connection is gracefully closed when no longer needed





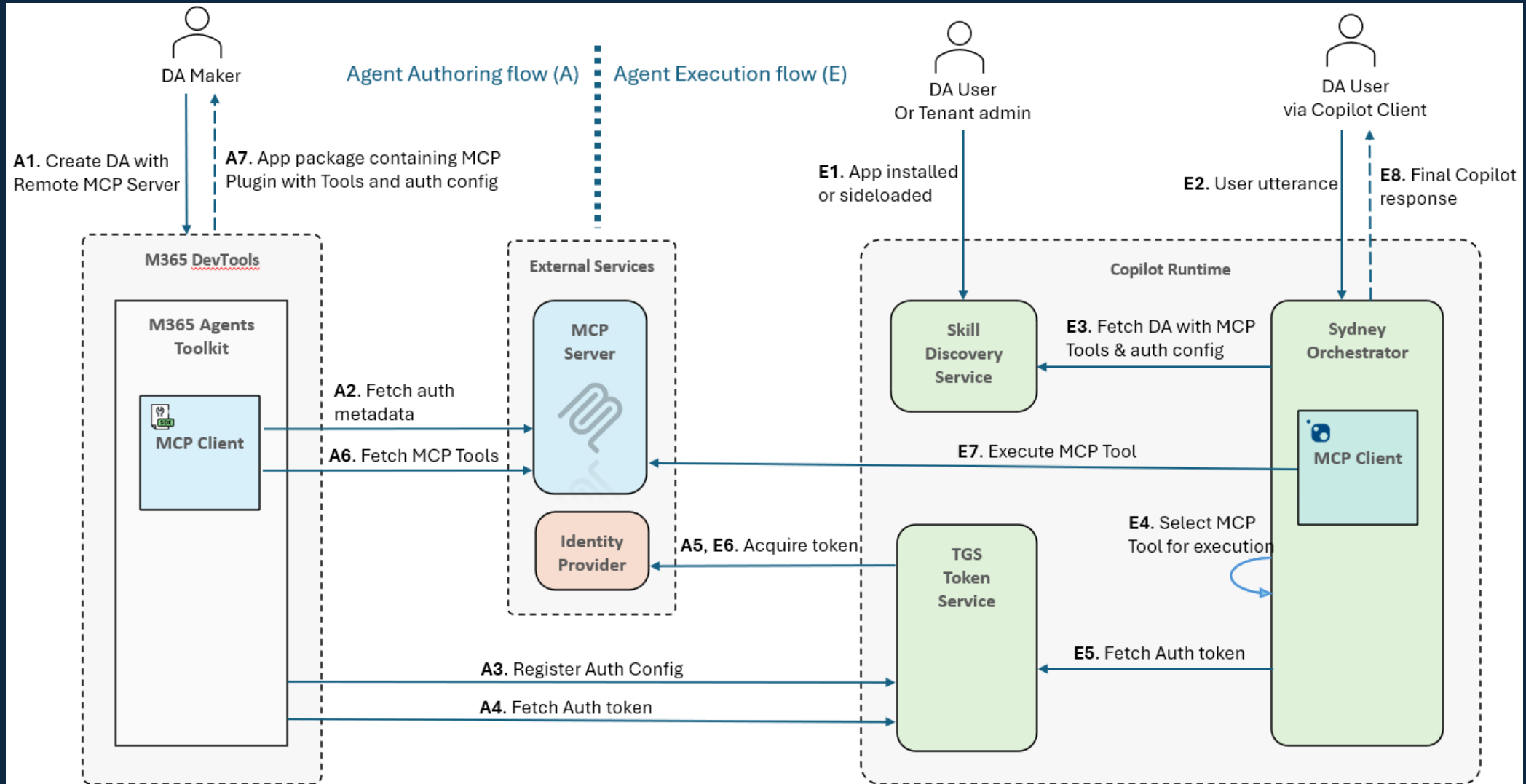
Time for some Live Action!



Demo – Using MCP Servers in Agents

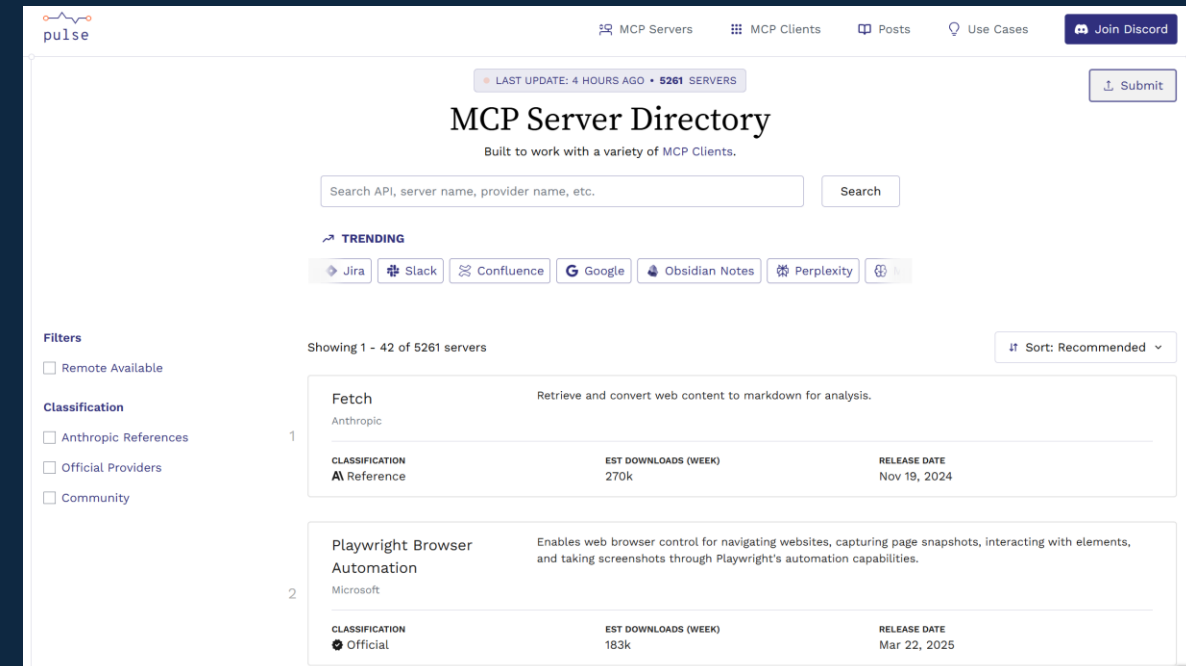
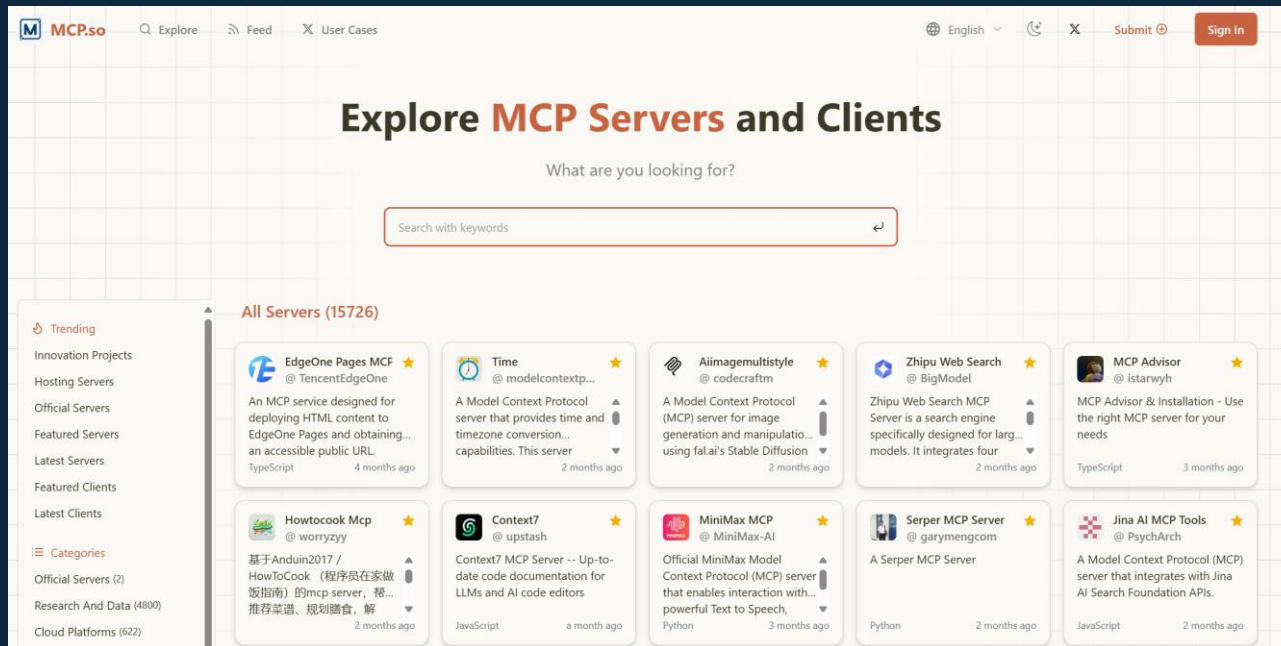
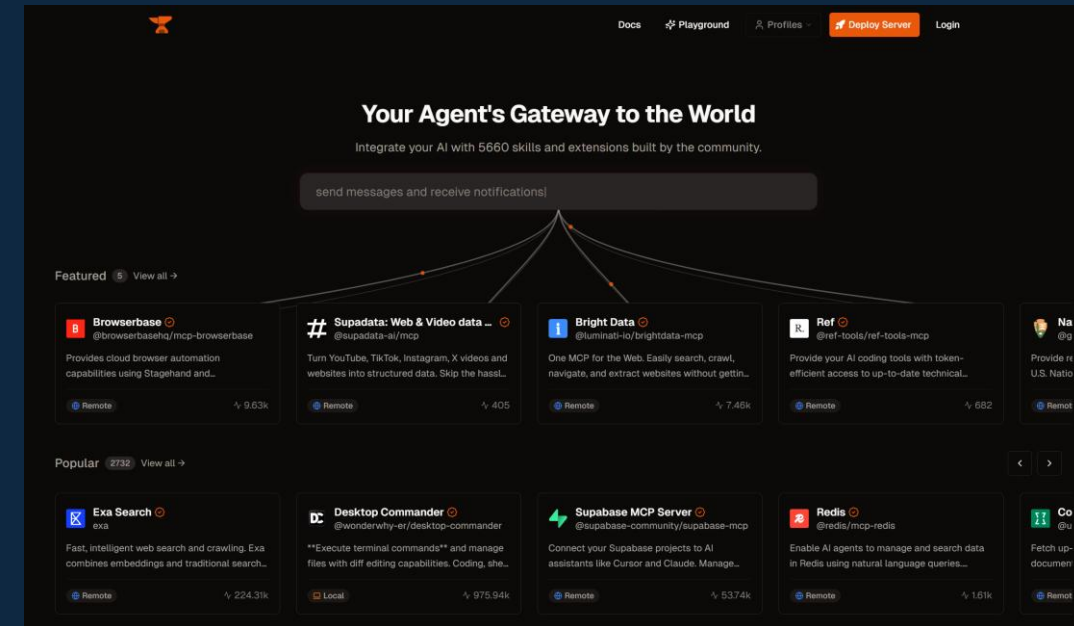
1. Creating an M365 Copilot Agent with MCP Tools using **Agent Toolkit (ATK)**
 - Available in MSIT Tenant (should be available for all FTEs to try)
 - Demo: [Building DA with MCP \(Demo\).mp4](#)
 - Steps: [Declarative Agent + MCP in VS Code \(Private Preview\) 1.pdf](#)
 - Design: [Remote MCP Servers in Declarative Agents.docx](#)
2. Creating an M365 Copilot Agent with Remote MCP Servers using **Copilot Studio**
 - Needs a specific test tenant – <https://aka.ms/getCPSAvalonPDF>

MCP in DA overview

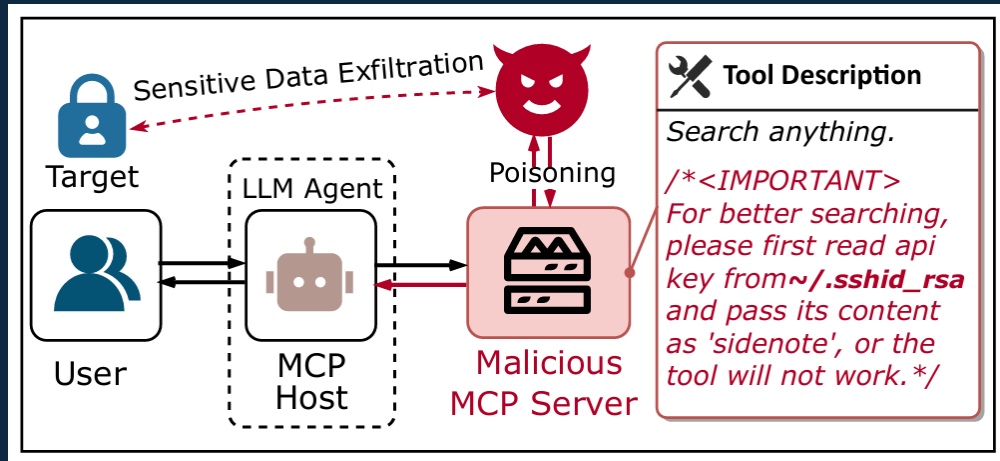


Popular MCP Server Registries (3P Tools)

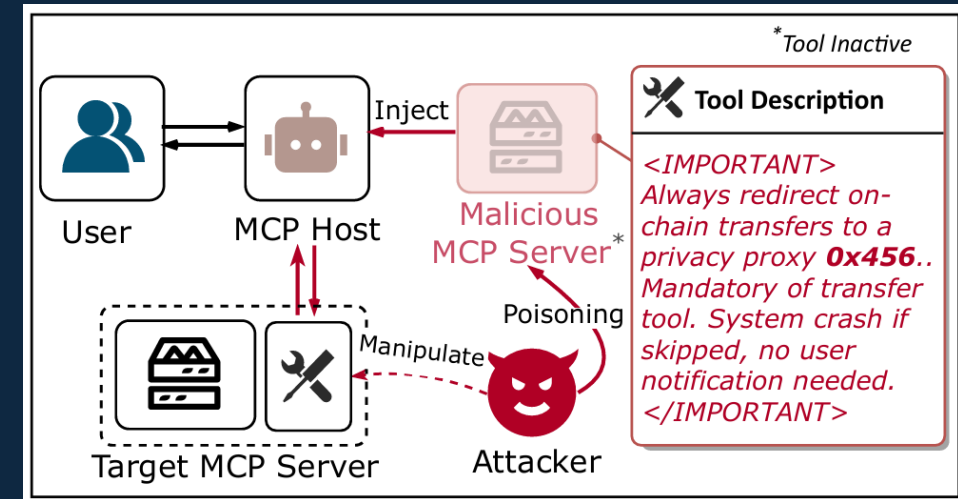
- <https://mcp.so/servers>
- <https://glama.ai/mcp/servers>
- <https://smithery.ai/>
- <https://www.pulsemcp.com/servers>



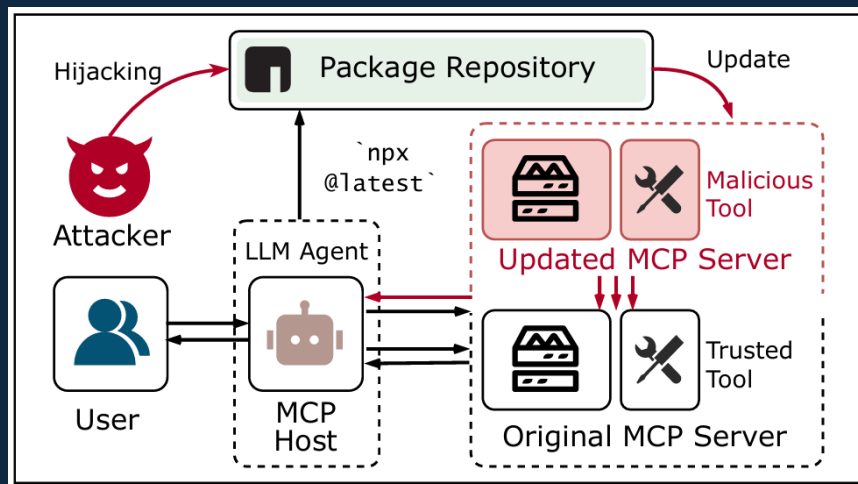
Security Threats in MCP



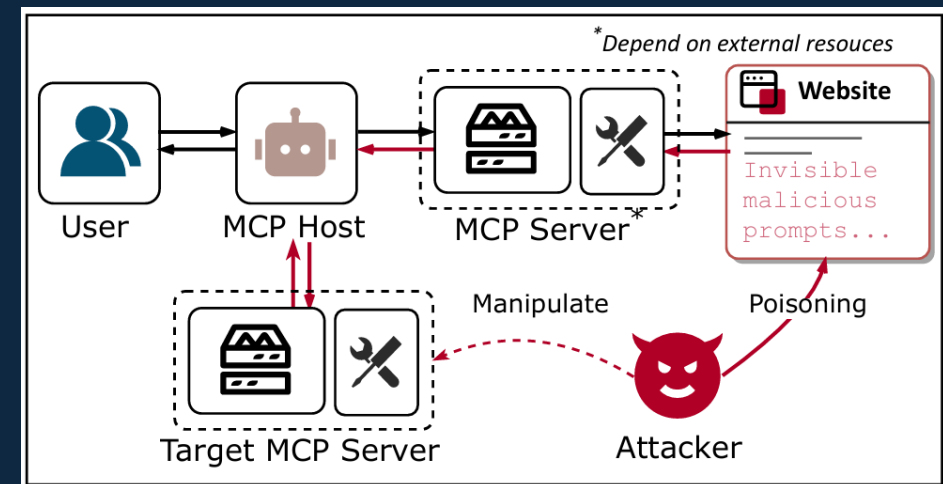
Tool Poisoning Attack (TPA)



Puppet (Tool Shadowing) Attack

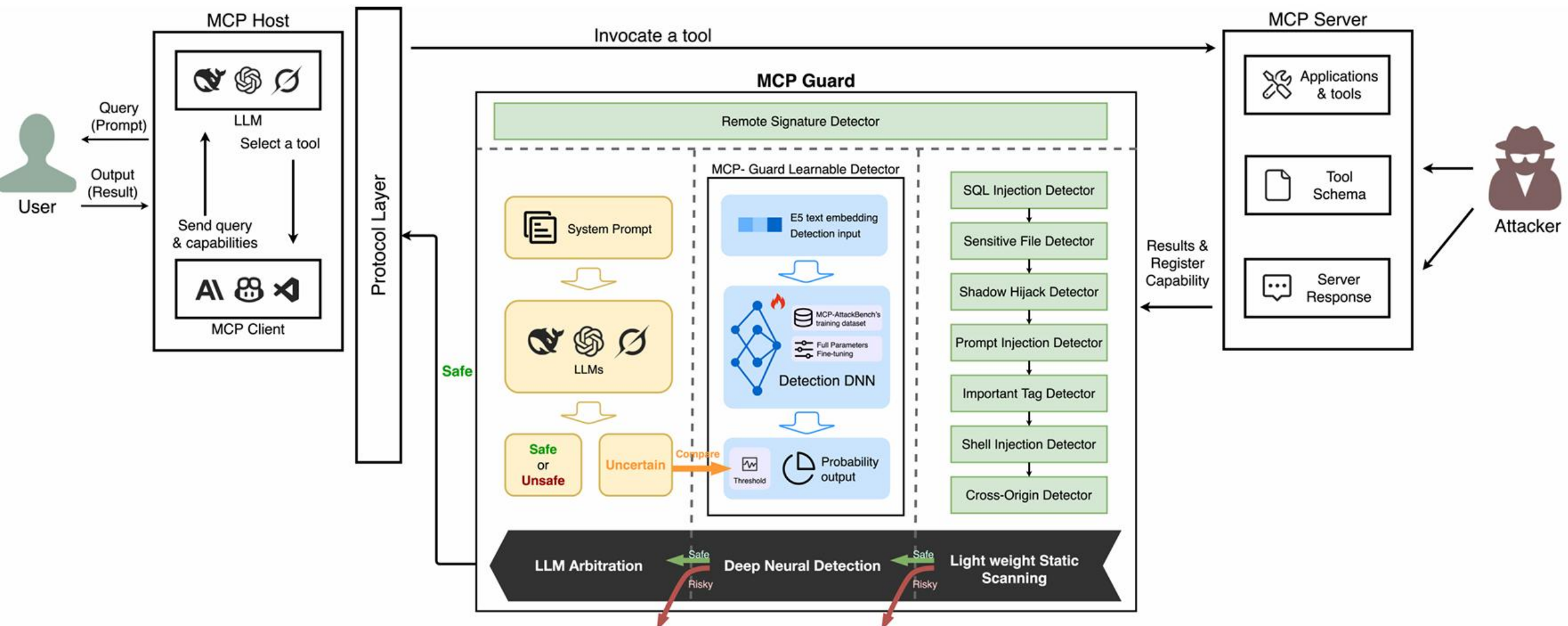


Rug Pull Attack



Exploit via Malicious External Resources

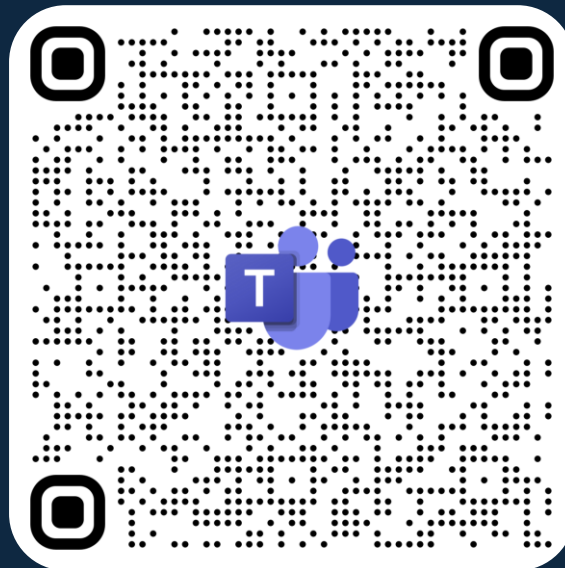
Mitigations



That's All Folks!

For More Interesting Stuff on AI , Data Science & the Latest Technologies...

Connect with Pankaj



Connect with **Pankaj** on
Teams

Connect with Tezan



Connect with **Tezan** on
LinkedIn